



KIU

KAATSU INTERNATIONAL UNIVERSITY

Machine Learning-Based Metamorphic Malware Detection System for Medium-Scale Enterprises

A Project Proposal by Mr. M. Daniel

Student ID - 11493

Supervised

By Ms. Praveeni Chethana

Mr. Malith Fernando

Contents

1.Introduction	4
1.1. Metamorphism.....	4
1.2. Opcodes	4
1.3. Entropy.....	4
2.Problem Statement	5
2.1. Obfuscation	5
3.Research Aim and Objectives	6
4.Scope and Deliverables.....	7
4.1. Executables (Malware Executables/ Legitimate Application Executables)	8
4.2. Disassembly of Opcodes.....	8
4.3. Entropy calculations	8
4.4. Extraction.....	8
4.5. Machine Learning model	9
5.Research Novelty.....	9
6.Literature Review	10
7.Proposed Methodology	11
7.1. Architectural Diagram	13
8.Tools & Resources Required.....	13
8.1. Isolated Analysis Environment	13
8.2. Hypervisor/Virtualization Software:.....	13
8.3. Analysis OS.....	14

8.4.	<i>Reverse Engineering & Feature Extraction.....</i>	14
8.5.	<i>PE Parsing Libraries:</i>	14
8.6.	<i>Dataset Resources</i>	14
8.7.	<i>Benign Repositories</i>	14
8.8.	<i>Machine Learning & Development Stack.....</i>	14
8.9.	<i>Data Processing & Evaluation</i>	14
9.	<i>Project Plan.....</i>	15
10.	<i>Reference</i>	16

1. Introduction

Malware, short for malicious software, refers to "programs covertly inserted into a system with the intent to compromise the confidentiality, integrity, or availability of the victim's data, applications, or operating system, or to otherwise disrupt or harm the victim [1].

1.1. Metamorphism

Metamorphism refers to approaches used to transform a piece of software into distinct instances [2]. Traditional antivirus fail to detect metamorphic malware due to variability in the internal structures [2]. Metamorphic malware is one of the most difficult types of malwares to deal with because it modifies its internal binary structure every time it replicates or runs [2].

The increasing reliability of digital systems has brought cybersecurity to the fore in every organization around the world. Enterprise networks are being targeted by frequent and sophisticated malware attacks, causing data breaches, ransomware, and system disruptions. Malware authors today use sophisticated evasion techniques, making traditional malware detection methods ineffective.[1].

1.2. Opcodes

Instructions opcodes as a statistical feature to check the similarity of the executables. We try to recognize a given file a morphed copy of another one. We think that while obfuscation functions change the visual structure of morphed variants of a virus, but many essential common features with these variants will be still remained, which contain the basic instructions and present their related functions [3]. Opcode sequences help in understanding the behavior of a program, and this is due to the fact that malicious code usually follows a specific logic in its execution, such as system modification, privilege escalation, or persistence installation [3].

1.3. Entropy

Considering that several anti-viruses normally perform statistical analysis using features extracted from the binary file, such as: n-grams, entropy, opcodes, signatures, size, imports section, or in specific kinds of malware like Android malware, permissions and intent actions, among others [4].

An entropy profile considers the binary representation of a file, divides it into chunks, calculates the Shannon entropy of these chunks, and aggregates them, forming an entropy sequence called the entropy profile [4]. These entropy profiles have shown a strong accuracy for deciding whether a piece of software is malware or not, even when the piece of malware is using metamorphic or polymorphic/packing-based concealment [4].

Malware continues to pose a significant threat to the security and operational integrity of medium-scale enterprises. As malware authors employ increasingly sophisticated evasion techniques, traditional detection methods—particularly those relying on single-feature static analysis—struggle to maintain high detection rates and operational efficiency. In response, hybrid machine learning-based frameworks that integrate opcode static analysis and entropy-based statistical analysis have emerged as promising solutions. These frameworks aim to leverage the complementary strengths of both feature types to improve detection rate, F1-score, and computational efficiency, especially when evaluated on publicly available malware datasets relevant to enterprise environments.

2. Problem Statement

It is estimated that over 66% of malware are obfuscated variations of previously known threats [5]. Polymorphic variants are becoming increasingly common, having increased from 50% to 96% of detected and blocked threats by Windows Defender from 2010 to 2017 [6]. Metamorphic malware, which modifies its internal structure while maintaining the same behavior, presents the greatest challenges among obfuscated malwares.

2.1. Obfuscation

Metamorphic malware uses complex transformation techniques that are difficult to avoid by detection systems [7].

The evasion techniques include:

- I. Dead-Code Insertion: Dead-code insertion, or some-times referred to as garbage code insertion, is an obfuscation technique which inserts byte code sequences into a binary without affecting functionality [7]

- II. Registry Reassignment: Registry Reassignment, or sometimes referred to as Registry Renaming, is an obfuscation technique which swaps unused registers or memory variables with those currently used by the program [7] .
- III. Instruction Substitution: The instruction-substitution technique introduces an additional layer of obfuscation on the existing techniques discussed. The power of instruction-substitution comes from the fact that there is a seemingly endless diversity to the substitutions you can introduce to an existing instruction framework [7].
- IV. Code Transposition: Code transposition, or sometimes called instruction permutation, is an obfuscation technique which utilizes conditional or unconditional jmp statements to reorder single or blocks of instructions [7].

The above transformations are applied to a binary in a manner that maintains its malicious characteristics. Signature-based antivirus software is no longer effective due to the different binary signatures generated by each piece of malware [8]. Malware analysis tools that use dynamic analysis run the malware in a virtual environment, but this is resource-intensive and may not work if the malware is able to detect a sandbox environment [9].

For medium-scale enterprises, they usually don't possess cybersecurity infrastructure, sophisticated monitoring systems, or specialized teams for malware analysis [10] .

Hence:

1. Unknown malwares are not detected.
2. Network compromise is spreading.
3. Downtime is increasing.

Thus, it is necessary to develop an intelligent automated detection framework that can recognize metamorphic malware without executing them.

3. Research Aim and Objectives

The main goal of this research is to develop an intelligent machine learning-based metamorphic malware detection model that combines both opcode static analysis and entropy based statistical analysis in the accurate detection of malicious executable files in target environments.

Metamorphic malware is one of the most challenging malware types for modern-day cybersecurity defense to deal with since it can change its internal binary structure using various

techniques like instruction substitution, code permutation, register reassignment, and junk code insertion [7]. These changes enable malware variants to evade traditional signature-based antivirus detection systems, thereby creating a risk for the organization. Therefore, the proposed system attempts to address the shortcomings of traditional malware detection systems by using a data-driven approach that can effectively detect new malware variants. The research aims to build a hybrid feature extraction framework that is able to analyze an executable file without executing it, hence reducing the risk of operation in dynamic analysis.

The proposed framework will utilize opcode extraction in obtaining the execution behavior of a program [3], while entropy analysis will help in identifying abnormal randomness characteristics that are often introduced in a program due to encryption, packing, and/or obfuscation techniques [11]. Another significant goal that this research hopes to accomplish is the assurance that the proposed solution will continue to be computationally efficient in the context of medium-scale enterprise infrastructures that might not necessarily have advanced security systems in place [10].

The use of optimized machine learning algorithms is aimed at ensuring that the proposed solution can classify with high accuracy while ensuring that the processing overhead is minimal. In response, hybrid machine learning-based frameworks that integrate opcode static analysis and entropy-based statistical analysis have emerged as promising solutions [1], [11]. These frameworks aim to leverage the complementary strengths of both feature types to improve detection rate, F1-score [12], and computational efficiency, especially when evaluated on publicly available malware datasets relevant to enterprise environments. F1-score demonstrated the most reliable and balanced performance across all scenarios [12].

4. Scope and Deliverables

The scope and deliverables of the research is to find novel methodology with the aim of achieving the target with machine learning-based metamorphic malware detection model through a well-defined structured step. This will help to efficiently identify enterprise-class malware detection mode. Dataset The first objective is to develop an extensive labelled dataset that includes both malicious and benign executable files. The accuracy of machine learning models depends on the quality of training data the being used in training period of the model [13].

Due to these reasons the dataset for the project will encompass malware samples collected from various publicly available threat intelligence sources and cyber security research datasets. Similarly, data will be collected from legitimate software applications.

4.1. Executables (Malware Executables/ Legitimate Application Executables)

The second objective executable files in the dataset will be accurately labeled as malicious or benign by machine learning model. The dataset will be representing different kinds of malware samples to ensure the generalization capabilities of the developed model. Proper data validation methods will be used to filter and extract correct data and remove corrupted data [14].

4.2. Disassembly of Opcodes

The third objective is to translate the executable binary files into assembly language instructions using static disassembly techniques. Reverse engineering software like Ghidra or other disassemblers will be employed to analyze binary code structures and extract assembly language instructions from executable binaries. The disassembly technique will enable the analysis of program instructions without the dangers of dynamic execution [1], [15], [2].

4.3. Entropy calculations

To implement an entropy calculation for your Machine Learning-based system, you should utilize Shannon Entropy to quantify the "randomness" of bytes within the executable's sections [4], [9], [11], [16]. Obfuscation Detection: High entropy values () flag "high randomness" blocks typical of encryption or packing used to hide malicious opcode patterns. Signature Mapping: Calculating entropy across PE sections (e.g., .text) creates a unique "structural signature" of the file's information density.

ML Integration: These scores serve as key features for your XGBoost meta-learner, distinguishing between standard code (low entropy) and malicious scrambled payloads (high entropy).

4.4. Extraction

The fifth objective is centered on the extraction of the opcode sequence from the disassembled assembly code. Opcodes represent the processor activities that include data movement, function calls, branching instructions, as well as memory manipulation activities [1], [2], [15].

4.5. Machine Learning model

At the end the model will be trained using extracted and validated data with correct-by-correct bias and variance with suitable optimized points to give the hypothetical result.

5. Research Novelty

The novelty of this research is based on the development of a hybrid machine learning-based metamorphic malware detection framework using opcode behavioral analysis and entropy-based statistical analysis to address the limitations of traditional malware detection mechanisms. Current malware detection systems are mostly based on signature-based detection mechanisms or individual feature extraction mechanisms, which are not effective against continuously changing metamorphic malware types. Metamorphic malware uses sophisticated evasion techniques such as instruction substitution, register reassignment, junk code injection, and control flow restructuring [7].

These techniques change the binary appearance of malware but retain their malicious capabilities. Traditional antivirus software cannot detect new types of malwares because of their dependency on previously known signatures or hash values [17]. The proposed research will present a novel hybrid feature engineering approach that considers both execution behavior and structural randomness in executable files. The extraction of opcodes will help in analyzing processor-level operation patterns derived from binary code, facilitating the recognition of malicious behavioral characteristics among structurally different types of malwares.

Entropy analysis will also help in identifying abnormal randomness levels caused due to encryption, packing, or obfuscation techniques commonly employed in metamorphic types of malwares [16]. Unlike existing research that considers both opcode analysis and entropy detection independently, this research will incorporate both types of feature categories in a single feature vector representation. This will help in overcoming some of the limitations of existing approaches that rely solely on a single feature model.

The proposed approach will utilize both opcode and entropy measurements, where opcodes will help in identifying behavioral characteristics and entropy will help in identifying concealed payload techniques.

The primary novelty of this research is the development of a dual-channel static detection framework that treats opcode sequences and structural entropy as a singular, unified feature vector for identifying metamorphic malware. Unlike traditional methods that rely on dangerous dynamic execution or isolated feature sets, this work demonstrates that a purely static hybrid approach can capture the behavioral intent of a program while simultaneously identifying its structural concealment [9].

This research makes three unique contributions to the field: Static Synergy of Features: The novelty of this work is in the simultaneous extraction of static opcodes for mapping malicious logic and static entropy for detecting hidden payloads without executing the file, thus removing the operational risks of dynamic sandboxes. Architectural Fusion (Stacked MetaLearner): Although prior studies like [15] have utilized these features, the novelty of this research is in the introduction of a Stacked Generalization Meta-Learner using XGBoost for correlating static randomness with instruction logic, thus reducing the False Positive Rate (FPR) [12] associated with single-feature models. Enterprise-Centric Design: The novelty of this work is in its focus on static triage prior to execution, thus making it uniquely optimized for medium-scale enterprises who lack the monitoring infrastructure and forensic teams needed for dynamic analysis of high overheads.

6. Literature Review

The growing pace of cyber threats has led to a substantial rise in the need for intelligent malware detection systems [1], [12], [18]. Conventional antivirus tools are based on signature detection methods they are fundamentally lacking the ability of mitigating zero-day exploits and metamorphic malware variants. These critical vulnerabilities necessitate the development of behavior-based detection frameworks [1], [4], [15], [19].

Metamorphic malware is one of the most complex types of malwares because it has the capability to transform its own code into new code with the same malicious functionality. The attacker uses various methods such as instruction substitution, control flow manipulation, register assignment, and junk code insertion to produce new binary variants [2], [3], [7].

Commercial cyber security vendors have developed heuristics and behavioral detection methods to identify the vulnerabilities of signature-based detections. However, heuristic rules highly depend on manual tuning and require frequent updates by security analysts [20]. These factors lead researchers to investigate machine learning-based approaches for malware detection. Malware analysis techniques can be divided into static and dynamic analysis. In the dynamic analysis technique, the malware is executed in sandbox environments [21]. Despite the advantage of obtaining detailed behavioral knowledge, the technique is associated with some challenges, including the high computational costs and slow speeds. Moreover, the technique is vulnerable to evasion attacks, especially in the detection of advanced malware [21]. Advanced malware can identify sandbox environments and halt malicious behavior. In the static technique, the malware is analyzed without executing the file. The technique is safe and efficient. Using the technique, malware can be converted into assembly instructions through the disassembly process.

The effectiveness of machine learning in detecting malware has been proven in existing research; however, its limitation in handling complex metamorphic malware using individual feature-based methods is a problem. Static opcode analysis offers behavioral characteristics, and entropy analysis offers obfuscation characteristics [9], [21]. The combination of both methods is a promising approach in improving detection accuracy while ensuring scalability and computational efficiency.

7. Proposed Methodology

The research undertakes a structured experimental research approach to design and develop a machine learning-based metamorphic malware detection system using static analysis techniques. The research undertakes a quantitative research approach with the help of supervised learning techniques to classify executable files as malicious or benign.

The research approach starts with the collection of the data set consisting of malware and benign files collected from publicly available cybersecurity research repositories and trustworthy sources of legitimate executable files. The data preprocessing activities such as validation, duplicates removal, and labeling will be undertaken to make the research reliable and trustworthy [14].

The next stage will be static binary disassembles to transform executable files into instructions written in assembly language without running untrusted executable files. This approach will be safer and more scalable than dynamic sandbox execution, which may consume substantial computational power [7]. The reverse engineering technique used to transform binary instructions into assembly language. The disassembled output will be used to identify sequences of opcode instructions related to processor operations.

The opcode instructions represent program behavior by describing actions taken during execution, including function calls, memory operations, and control flow execution [22]. The preprocessed stage will remove operands and memory operations to retain operational instructions. Opcode frequency will be determined and converted to numerical representations to help machine learning identify similarities between metamorphic malware despite their structural changes [3]. Besides opcode behavioral analysis, entropy calculation helps in detecting obfuscation techniques used by advanced malware. Shannon entropy theory is applied to measure randomness within executable files and their internal sections.

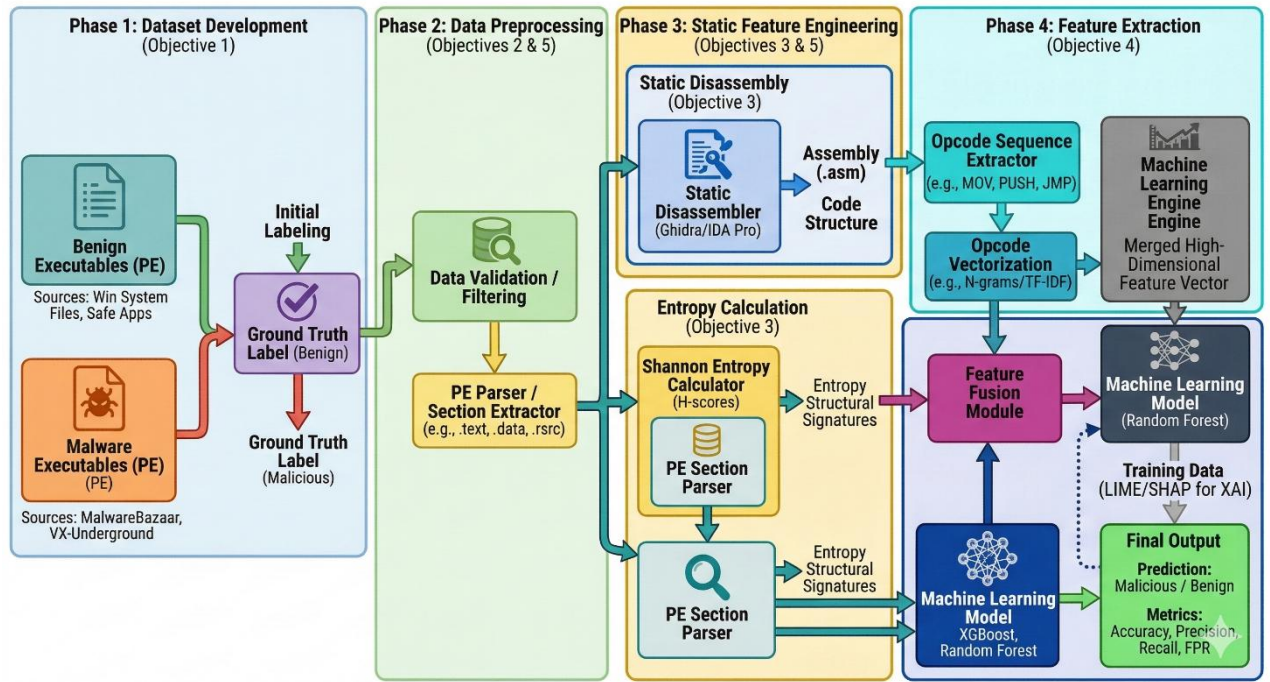
The model commonly focuses on encryption, compression, or packing techniques used malicious payloads to reveal by looking at abnormal entropy values [16]. Both global file entropy and section-level entropy will be computed to identify suspicious patterns. The extracted opcode features and entropy measurements will then be combined into hybrid feature vectors through a feature engineering process.

Data normalization, scaling, and dataset balancing techniques will serve to improve learning performance and reduce bias [14]. This hybrid feature approach permits the system to capture both behavioral intent and structural randomness, addressing weaknesses associated with single-feature malware detection models.

The final step in the proposed methodology is the development, evaluation, and deployment of the machine learning model in the development of the automated prediction system. Supervised classification techniques such as the Random Forest classifier and the Support Vector Machine classifier will be implemented in the proposed model. The training of the algorithm by feeding created data sets and evaluated using cross-validation techniques to tune the parameters of the model. The model's performance will be evaluated using accuracy, precision, recall, F1-score, and ROC curve methods to achieve accurate classification of the data while avoiding false positives [12]. The model will then be used in the development of the automated prediction system that can analyze the unknown executable files using disassembly, feature extraction, and

classification techniques. The proposed model can be used in the detection of malware in the near future in medium-scale enterprise networks while considering the recent adversarial malware behaviors observed in the cybersecurity intelligence frameworks such as the MITRE ATT&CK framework [23].

7.1. Architectural Diagram



8. Tools & Resources Required

8.1. Isolated Analysis Environment

To safely store and analyze potentially malicious executables without compromising the host network, a strictly isolated analysis environment must be set up using virtualization.

8.2. Hypervisor/Virtualization Software:

Oracle VirtualBox or VMware Workstation must be used to run isolated virtual machines on a host-only network.

8.3. Analysis OS

A sandboxed Linux distro, such as Kali or REMnux, and a Windows 10/11 setup for native PE file parsing are required

8.4. Reverse Engineering & Feature Extraction

Static Disassembly: Ghidra, developed by the NSA, must be used as the primary reverse engineering tool to statically convert executables into human-readable x86 assembly code.

8.5. PE Parsing Libraries:

To programmatically parse the PE headers and pull out section-level information (.text, .rdata, etc.), Python tools such as PEfile must be used. For the math, Python libraries such as SciPy or NumPy must be used to compute Shannon entropy on the extracted byte sequences.

8.6. Dataset Resources

Malware Repositories: Public threat intelligence platforms such as MalwareBazaar, VirusShare, or VX-Underground must be used to gain access to a diverse set of in-the-wild metamorphic malware.

8.7. Benign Repositories

Clean, authentic executables must be sourced from fresh Windows OS installations or software repositories such as Portable Apps.

8.8. Machine Learning & Development Stack

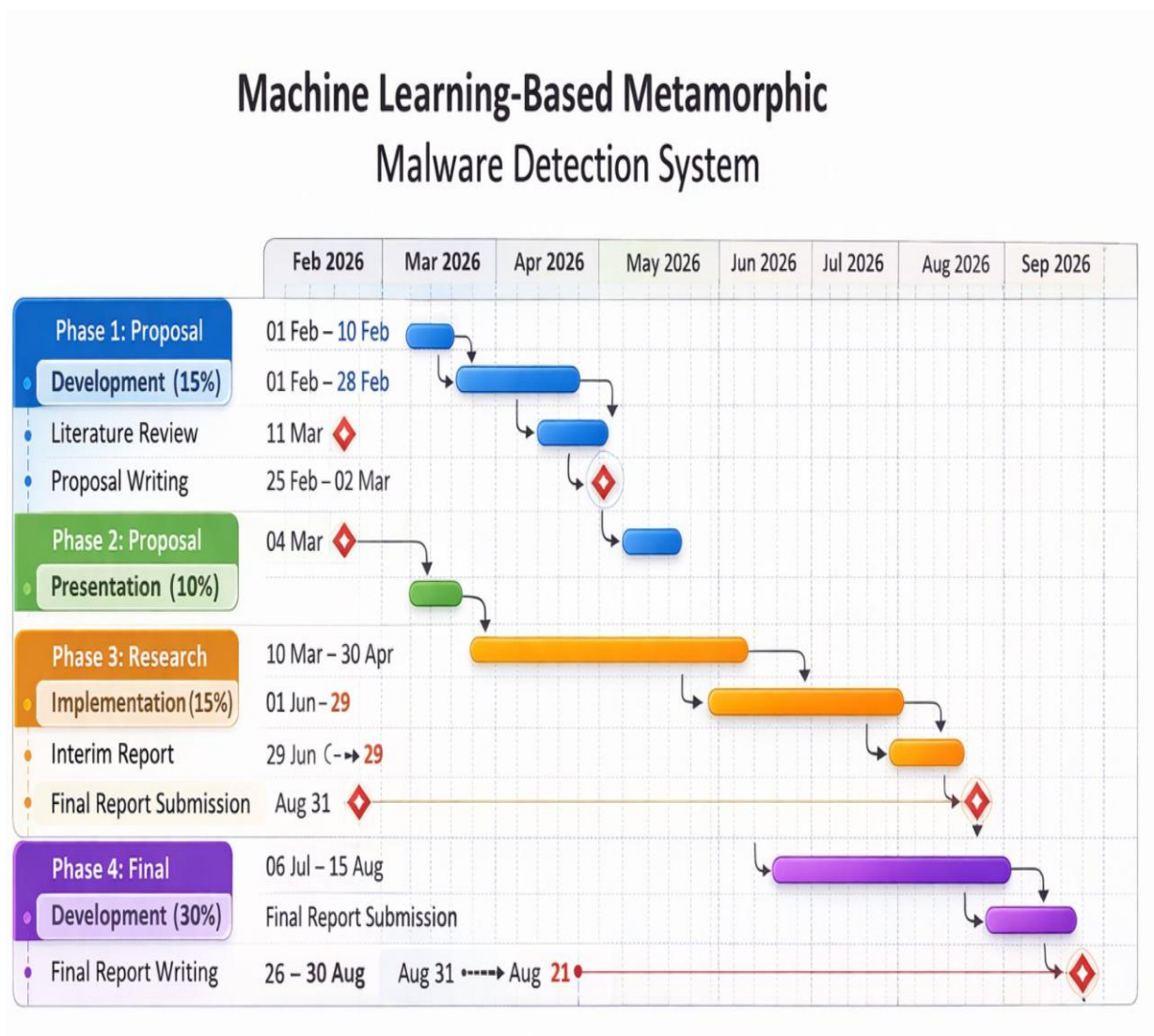
Programming Language: Python 3.x must be used as the programming language.

Primary ML Framework: XGBoost must be used to develop the meta-learner model for classifying the combined opcode and entropy features.

8.9. Data Processing & Evaluation

Scikit-learn must be used for dataset splitting, cross-validation, and final evaluation metrics, whereas Pandas must be used for data management.

9. Project Plan



Phase & Weight	Task / Deliverable	Timeline
Phase 1: Proposal Development	Initial Proposal Structuring	01 Feb – 10 Feb
	Extended Development	01 Feb – 28 Feb
	Proposal Writing	25 Feb – 02 Mar
	Literature Review	11 Mar
Phase 2: Proposal Presentation	Proposal Defense Presentation	04 Mar
Phase 3: Research Implementation	Core Research Phase	10 Mar – 30 Apr
	System Implementation	01 Jun – 29 Jul
	Interim Report	29 Jun
	Final Report Submission	31 Aug
Phase 4: Final Development	Final Development Polish	06 Jul – 15 Aug
	Final Report Writing	26 Aug – 30 Aug

10. Reference

- [1] “Metamorphic Malware Detection with Machine Learning.”
- [2] J. Kuriakose, “Unknown Metamorphic Malware Detection: Modelling with Fewer Relevant Features and Robust Feature Selection Techniques,” 2015.
- [3] B. B. Rad and M. Masrom, “Metamorphic Virus Detection in Portable Executables Using Opcodes Statistical Feature”.
- [4] H. D. Menéndez and J. L. Llorente, “Mimicking Anti-Viruses with Machine Learning and Entropy Profiles,” *Entropy*, vol. 21, no. 5, p. 513, May 2019, doi: 10.3390/e21050513.
- [5] A. K. Jha, A. Vaish, and S. Patil, “A Novel Framework for Metamorphic Malware Detection,” *SN Comput. Sci.*, vol. 4, no. 1, p. 10, Oct. 2022, doi: 10.1007/s42979-022-01433-1.
- [6] C. Magazine, “The Devastating Effect Of Polymorphic Malware,” Cybercrime Magazine. Accessed: Mar. 07, 2026. [Online]. Available: <https://cybersecurityventures.com/the-devastating-effect-of-polymorphic-malware/>
- [7] K. Brezinski and K. Ferens, “Metamorphic Malware and Obfuscation: A Survey of Techniques, Variants, and Generation Kits,” *Secur. Commun. Netw.*, vol. 2023, no. 1, p. 8227751, 2023, doi: 10.1155/2023/8227751.
- [8] D. Lin, “Hunting for Undetectable Metamorphic Viruses,” Master of Science, San Jose State University, San Jose, CA, USA, 2009. doi: 10.31979/etd.zw8c-jybb.
- [9] N. Alshahwan, E. T. Barr, D. Clark, G. Danezis, and H. D. Menéndez, “Detecting Malware with Information Complexity,” *Entropy*, vol. 22, no. 5, p. 575, May 2020, doi: 10.3390/e22050575.
- [10] M. Awan, A. Alam, and M. Kamran, “Cybersecurity Challenges in Small and Medium Enterprises: A Scoping Review,” *J. Cyber Secur. Risk Audit.*, vol. 2025, no. 3, pp. 89–102, Jul. 2025, doi: 10.63180/jcsra.thestap.2025.3.7.
- [11] D. Baysa, “Structural Entropy and Metamorphic Malware,” Master of Science, San Jose State University, San Jose, CA, USA, 2012. doi: 10.31979/etd.6zrt-mb7y.
- [12] K. M. Sujon, R. Hassan, K. Choi, and M. A. Samad, “Accuracy, precision, recall, f1-score, or MCC? empirical evidence from advanced statistics, ML, and XAI for evaluating business predictive models,” *J. Big Data*, vol. 12, no. 1, p. 268, Dec. 2025, doi: 10.1186/s40537-025-01313-4.

- [13] S. Mohammed *et al.*, “The effects of data quality on machine learning performance on tabular data,” *Inf. Syst.*, vol. 132, p. 102549, Jul. 2025, doi: 10.1016/j.is.2025.102549.
- [14] S. Shankar, L. Fawaz, K. Gyllstrom, and A. Parameswaran, “Automatic and Precise Data Validation for Machine Learning,” in *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, in CIKM ’23. New York, NY, USA: Association for Computing Machinery, Oct. 2023, pp. 2198–2207. doi: 10.1145/3583780.3614786.
- [15] V. M. González-Gorrín, J. Prieto-Blázquez, and J. Arnedo-Moreno, “A Hybrid Machine Learning Intrusion Detection Method for Metamorphic Malware,” 2024, *SSRN*. doi: 10.2139/ssrn.5049115.
- [16] E. Radkani, S. Hashemi, A. Keshavarz-Haddad, and M. Amir Haeri, “An entropy-based distance measure for analyzing and detecting metamorphic malware,” *Appl. Intell.*, vol. 48, no. 6, pp. 1536–1546, Jun. 2018, doi: 10.1007/s10489-017-1045-6.
- [17] M. Campion, M. Dalla Preda, and R. Giacobazzi, “Learning metamorphic malware signatures from samples,” *J. Comput. Virol. Hacking Tech.*, vol. 17, no. 3, pp. 167–183, Sep. 2021, doi: 10.1007/s11416-021-00377-z.
- [18] Z. Markel, “Machine Learning Based Malware Detection”.
- [19] D. Gibert, C. Mateu, J. Planes, and J. Marques-Silva, “Auditing static machine learning anti-Malware tools against metamorphic attacks,” *Comput. Secur.*, vol. 102, p. 102159, Mar. 2021, doi: 10.1016/j.cose.2020.102159.
- [20] “Gartner Identifies the Top Cybersecurity Trends for 2026,” Gartner. Accessed: Mar. 07, 2026. [Online]. Available: <https://www.gartner.com/en/newsroom/press-releases/2026-02-05-gartner-identifies-the-top-cybersecurity-trends-for-2026>
- [21] O. Or-Meir, N. Nissim, Y. Elovici, and L. Rokach, “Dynamic Malware Analysis in the Modern Era—A State of the Art Survey,” *ACM Comput Surv*, vol. 52, no. 5, p. 88:1-88:48, Sep. 2019, doi: 10.1145/3329786.
- [22] X. Li *et al.*, “BMOP: Bidirectional Universal Adversarial Learning for Binary OpCode Features,” *Wirel. Commun. Mob. Comput.*, vol. 2020, pp. 1–11, Dec. 2020, doi: 10.1155/2020/8876632.
- [23] “MITRE ATT&CK®.” Accessed: Mar. 09, 2026. [Online]. Available: <https://attack.mitre.org/>